第1回：LLMイントロダクション

LLMアプリケーション開発 2026

外部の資料を使って答えるアプリを作る

今日のゴール

- LLMが文章を生成する仕組みを、自分の言葉で説明する
- モデル、チャットサービス、APIの違いを説明する
- 資料を渡す理由と、回答を確かめる必要性を理解する

今日はコードの実装を求めません。完成品を観察します。

今日の90分

時間	内容
10分	授業の全体像と前の授業とのつながり
25分	LLMの仕組みと使い方
10分	完成アプリの準備と画面の確認
30分	同じ質問を異なる方式で比較
15分	結果の共有・確認問題・提出

観察では、回答だけでなく、渡した資料も見ます。

13回の授業

回	内容
1〜3	LLMの基礎、API、Webアプリとの連携
4〜6	外部資料、キーワード検索、意味検索
7〜8	関数の呼び出し、評価、安全な運用
9〜13	自分の題材で企画・制作・改善

毎回、同じアプリの機能を増やします。

前の授業とのつながり

既習の技術	今回の使い道
HTML・CSS・JavaScript	質問フォームと回答の画面
FastAPI・JSON・fetch	ブラウザとサーバーの通信
SQLite・SQL	参照資料と架空の課題の保存
入力検証・textContent	不正な入力や出力への対策

新しく加えるのは、外部のLLM APIと知識の検索です。

LLMとは

Large Language Model：大規模言語モデル。

大量の文章などから言葉の関係を学習し、入力された文脈に続く
トークンを予測して文章を生成します。

質問への回答、要約、分類、文章の書き換えなどに利用できます。
自然な文章であることと、内容が正しいことは別に確認します。

言語モデルの簡単な歴史

時期	主な考え方
従来	直前の単語の並びから、次の単語の頻度を推定
2000〜2010年代	ニューラルネットワークで言葉の特徴を学習
2017年	Transformerを提案した論文が発表
2018年以降	大規模な事前学習を使うモデルが発展
2022年以降	対話形式のサービスが普及

Transformerの考え方

同じ「はし」でも、周りの言葉によって意味が変わります。

- 「料理を、はしを使って食べる」→「料理」「食べる」が手がかりになり、箸とわかる
- 「はしを渡って、川の向こうへ行く」→「渡って」「川」が手がかりになり、橋とわかる

Attentionは、文脈内の要素どうしの関係を計算する仕組みです。
この例では、「はし」と周りの言葉との関係が、意味を捉える手がかりになります。

この授業では数式や学習処理の実装には立ち入りません。
入力にどんな文脈を与えるかに注目します。

トークンと生成

トークンは、モデルが扱う文章の区切りです。
1文字や1単語と必ず一致するわけではありません。

1. 入力をトークンに分ける
2. 文脈から次のトークンの候補を計算する
3. 選んだトークンを文脈に加える
4. 終了条件まで繰り返す

「文章の続きを作る」仕組みを、質問への応答にも使います。

生成の途中で増える文脈

説明用に「今日は」「晴れ」「です」を順に選んだとします。

段階	次の候補を考えるための文脈
最初	入力された質問や指示
1回選択後	入力 ＋「今日は」
2回選択後	入力 ＋「今日は晴れ」
3回選択後	入力 ＋「今日は晴れです」

区切りは説明用です。実際のトークン分割や生成結果ではありません。
生成済みの内容も、次を生成する文脈に含まれます。

学習と推論

処理	何をするか	この授業
学習	データからモデルのパラメータを調整する	実施しない
推論	既に学習したモデルに入力を渡し、出力を得る	APIで利用

授業資料を質問に添付するだけでは、モデルを再学習しません。
今回の回答で参照する情報として渡します。

資料を添えた質問の例

説明用の架空資料：「図書室の開室時間は平日16時までです。」

今回送るもの	内容
参考資料	上の開室時間の案内
質問	図書室は何時まで開いている？
回答を確かめる基準	平日16時という条件が一致するか

次の質問でも必要なら、アプリが資料を再び入力に含めます。
この教材は1問ずつ処理し、前の会話を自動で引き継ぎません。

モデル・サービス・API

言葉	役割	例
モデル	入力から出力を生成する仕組み	GPT、Claude、Geminiなど
サービス	人が画面から利用する製品	チャット画面など
API	プログラムから利用する窓口	自作アプリからの呼び出し

画面にある検索・履歴・ファイル機能を、モデル自体の能力と混同しないことが大切です。

自作アプリが用意する機能

「質問に答える画面」を作るには、生成以外の処理も必要です。

利用者から見える機能	アプリが行う処理
質問の入力欄	入力を受け取り、長さを確認する
資料を使った回答	資料を取得して入力に含める
出典の表示	取得した本文とファイル名を表示する
失敗したときの案内	エラーを利用者向けの表示に変える

完成品で、これらが画面のどこにあるかを観察します。

代表的なモデル系列

系列	提供元	観察する点
GPT	OpenAI	APIの機能、用途ごとのモデル
Claude	Anthropic	APIの機能、用途ごとのモデル
Gemini	Google	入出力の種類、APIの機能

世代、利用条件、対応機能は変わります。公式資料で確認します。
順位を暗記するのではなく、用途に合わせて比べます。

この授業で使うモデル

文章生成と関数選択には gpt-6-luna を使います。
第2回からResponses APIで呼び出します。

第6回の文章のベクトル化には、別用途の
text-embedding-3-small を使います。

生成用モデルとEmbedding用モデルは、仕事が異なります。

得意なことと、確認が必要なこと

使い方	確認すること
長い文の要約	大切な条件を落としていないか
問い合わせの分類	あいまいな例をどう扱うか
コードの説明	実際のコードと一致しているか
授業案内の回答	資料に根拠があるか

もっともらしい誤答を、ハルシネーションと呼ぶことがあります。

回答の確かめ方

説明用の架空資料：「提出は金曜17時まで。形式はPDF。」

回答例	本文と照合した結果
金曜17時までにPDFで提出	日時と形式が一致
金曜中に提出	17時という条件が抜けている
土曜17時までにPDFで提出	日付が資料と一致しない

同じように自然な文章でも、資料との一致は別に調べられます。
この照合を、完成アプリの観察でも行います。

モデルが知らない情報

「このクラスの次の提出物は何ですか？」

モデルが一般的なプログラミング知識を持っていても、
このクラスの案内を知っているとは限りません。

- 資料を渡す
- 必要な情報をDBから取得する
- 根拠がないときは確認できないと伝える

このための仕組みをWebアプリ側に作ります。

アプリの役割分担

場所	仕事
ブラウザ	質問と方式を送信し、結果を表示
FastAPI	入力を確認し、資料や関数を選ぶ
SQLite	資料や課題を保存
LLM API	渡された内容から回答や関数の引数を生成

LLMへ渡す情報と、実行できる操作をアプリ側が決めます。

完成アプリの起動

ForkしたリポジトリのCodespacesで、新しいターミナルを開きます。

```
source .venv/bin/activate
cd session01/exercise
python main.py
```

「ポート」から8000番を開き、可視性はPrivateのまま使います。
終了はCtrl+C。ポートが使用中なら前のサーバーを止めます。

画面で見る場所

- 動作モードはデモかAPIか
- 「回答の方式」を変更する
- 同じ質問で回答を比較する

下へスクロールすると、
参照資料と学習用の詳細欄があります。

右はデモで「資料なし」を選んだ実画面です。



デモとAPIで観察できること

方式	デモ	API
資料なし	固定の説明文	モデルが回答を生成
資料全体	取得した本文の抜粋	資料を入力に含めて生成

デモでは画面と資料の流れを確認します。生成品質は採点しません。
APIが使えない場合は、講師の実演を観察して記録します。
実演もできなければ、回答品質は未検証と記録します。

観察前の予想

質問：「第7回の提出物は？」

1. 資料なしで、モデルはこの教材の案内を確認できるか
2. 資料を渡した場合、どの本文と回答を比べればよいか

ワークシートに予想を1文ずつ書きます。
ここで扱う案内は、アプリの動作確認用の架空データです。
実際の授業の提出物とは区別します。

観察1：資料なし

「資料なし」を選び、「第7回の提出物は？」を送ります。

- デモ：固定文に「デモ」と表示されることを確認
- API：具体的な提出物を答えたか、分からないと答えたかを記録
- どちらも、授業案内の本文を渡していないことを確認

具体的な答えが返った場合も、案内と一致するかを後で照合します。
講師の実演を見る場合は、観察したモードも記録します。

観察2：資料全体

質問を変えず、「資料全体を渡す」に変更します。

1. 「LLMに渡した資料」から「第7回の提出物」を探す
2. 原本の `data/02-course.md` と内容を比べる
3. APIの場合は、回答の提出物と資料IDを本文と照合する

デモの場合は、生成文ではなく本文の抜粋が表示されます。
表示の違いを、そのままモデルの性能差と判断しません。

観察3：資料にない質問

「資料全体」で「来年度の試験日は何月何日？」と質問します。

この教材の資料には、来年度の具体的な試験日はありません。

- API：確認できないと答えたか、日付を作ってしまったか
- デモ：抜粋した本文だけで、その日付を確認できるか

答えのない質問での振る舞いも、アプリを評価する材料です。

比較結果の記録

`exercise/worksheet.md` に、実際に見た結果を記入します。

質問・方式	記録する内容
提出物・資料なし	応答内容、根拠を渡していないこと
提出物・資料全体	回答または抜粋、根拠の見出し
試験日・資料全体	日付の断定か保留か、未検証の項目

モードと、講師の実演か自分の操作かも残します。
予想と違った点を、隣の人に1つ説明します。

確認問題

1. 自然な文章なら、その内容は必ず正しい？
2. 資料を質問に添えたら、モデルを学習し直したことになる？
3. このアプリで資料を選んで入力に含めるのは誰？
4. デモの抜粋表示から、モデルの生成品質を評価できる？

回答と理由をワークシートへ記入します。

提出物と次回

提出：Forkした自分のリポジトリと、
`session01/exercise/worksheet.md` のGitHub URL。

ワークシートに、観察したモード・比較結果・確認問題の回答を残します。

次回はPythonから gpt-6-luna のAPIを呼び出します。